

Séance 4 — Visualisation et instrumentation

Introduction à la programmation Python — IUT 1re année

Support de cours

Séance 4

Visualisation et instrumentation

Pourquoi visualiser des données ?

Un graphique révèle en un coup d'œil des tendances, des anomalies ou des seuils dépassés qu'un tableau de chiffres cache facilement. En métrologie, c'est indispensable pour valider une série de mesures.

Qu'est-ce que Matplotlib ?

Matplotlib est la bibliothèque de référence pour tracer des graphiques en Python.

```
import matplotlib.pyplot as plt # alias conventionnel
```

Graphique en courbes

```
temps = [0, 1, 2, 3, 4] temperature = [20.1, 20.4, 20.9, 21.5, 22.0]
plt.plot(temps, temperature) plt.show()
```

Nuage de points et histogramme

```
# Nuage de points : relation entre deux variables plt.scatter(temps, temperature)
plt.show() # Histogramme : distribution d'une série de valeurs mesures = [12.1,
12.5, 12.4, 12.6, 12.3, 12.9] plt.hist(mesures, bins=5) plt.show()
```

Personnaliser un graphique

```
plt.plot(temps, temperature, color="red", linestyle="--", marker="o", label="Sonde
1") plt.xlabel("Temps (s)") plt.ylabel("Température (°C)") plt.title("Évolution de
la température") plt.legend() plt.grid(True) plt.show()
```

Figures, axes et sous-figures

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4)) # 1 ligne, 2 graphiques
axes[0].plot(temps, temperature) axes[0].set_title("Courbe") axes[1].hist(mesures,
bins=5) axes[1].set_title("Histogramme") plt.tight_layout() plt.show()
```

Exporter et sauvegarder

```
plt.plot(temps, temperature) plt.savefig("courbe_temperature.png", dpi=150)
```

Notions d'instrumentation

- **Instrument** : appareil de mesure (capteur + électronique + interface)
- Communication **matériel** ↔ **logiciel** : l'instrument envoie/reçoit des données via un port (série, USB, réseau...)
- **Liaison série** : transmission de données bit par bit, très répandue pour dialoguer avec des cartes comme l'Arduino

Le protocole SCPI

SCPI (Standard Commands for Programmable Instruments) est un langage de commandes textuelles standardisé pour piloter des instruments de mesure (multimètres, oscilloscopes...).

```
*IDN? -> identifie l'instrument MEASure:VOLTage:DC? -> demande une mesure de
tension continue
```

Port série sur un PC et encodage

- Sous Windows : COM3, COM4... ; sous Linux : /dev/ttyUSB0, /dev/ttyACM0...
- Paramètres à faire correspondre entre les deux appareils : **vitesse (bauds)**, bits de données, parité
- **Encodage des caractères** : les données brutes sont des octets ; il faut les décoder (souvent en `ascii` ou `utf-8`) pour obtenir du texte lisible

Utiliser pyserial

```
import serial
port = serial.Serial("COM3", baudrate=9600, timeout=1) # ouvre la
liaison
ligne = port.readline() # lit une ligne (bytes)
texte = ligne.decode("utf-8").strip() # décodage en texte
print(texte)
port.write(b"MEASURE\n") # envoie une commande (bytes)
port.close()
```

Exemple : lire un capteur Arduino

```
import serial import time port = serial.Serial("COM3", baudrate=9600, timeout=1)
time.sleep(2) # laisser le temps à l'Arduino de redémarrer for _ in range(10):
ligne = port.readline().decode("utf-8").strip() if ligne: valeur = float(ligne)
print(f"Mesure reçue : {valeur}") port.close()
```

À vous de jouer

Direction les exercices de la séance 4 →

Exercices

Exercices — Séance 4 : Visualisation et instrumentation

Ex. 1 — Première courbe À partir du CSV de la séance 3, tracer température en fonction du temps avec `plt.plot`, en ajoutant titre et légendes des axes.

Ex. 2 — Nuage de points et histogramme Sur un même script, tracer un nuage de points (deux séries de mesures liées) puis un histogramme d'une série de valeurs.

Ex. 3 — Personnalisation et export Reprendre l'Ex. 1 : changer couleur/style de ligne, ajouter une grille et une légende, puis exporter le graphique en `.png`.

Ex. 4 — Détection de dépassement de seuil Ajouter au graphique une ligne horizontale représentant un seuil (`plt.axhline`), et faire apparaître en rouge les points qui le dépassent.

Ex. 5 — Sous-figures Créer une figure avec 2 sous-graphiques côte à côte (courbe + histogramme) pour la même série de mesures, avec `plt.subplots`.

Ex. 6 — Lecture d'un port série (si matériel disponible) Avec `pyserial`, lister les ports disponibles, se connecter à une carte (Arduino ou simulateur), lire 10 mesures, les stocker dans une liste puis les tracer.